👩‍🏫 RAMÓN ABRAMO • GUÍA TÉCNICA DETALLADA

Versión 1: Lógica Básica Lineal

Desglose pedagógico de la solución elemental con variables globales, bucles clásicos y manipulación directa del DOM.

> La **Versión 1** constituye el punto de entrada natural para cualquier estudiante que aborda por primera vez la lógica de un videojuego en JavaScript. Elimina complejidades avanzadas de programación orientada a objetos o patrones reactivos para centrarse estrictamente en los fundamentos: variables, arrays, bucles `for` y condiciones `if/else`.

Estructura de la Carpeta `version-1-basica-lineal/`

Archivo	Propósito	Destinatario
`index.html`	Estructura semántica con SVG del ahorcado, display y campo de texto.	Alumno / Docente
`main.css`	Estilos visuales, contraste moderno y clases `.visible` para el SVG.	Alumno / Docente
`app.js`	Código maestro con explicaciones exhaustivas hito por hito.	Docente
`app.alumnos.js`	Plantilla con esqueletos y bloques `TODO` listos para completar.	Alumnado
`app.solucion.js`	Solución canónica limpia sin comentarios extensos para corrección rápida.	Docente
`presentacion.html`	Diapositivas interactivas proyectables en el aula para explicar la V1.	Docente en Aula

Metodología de Trabajo en el Aula

Recomendación para Ramón Abramo

1. Proyecta en la pizarra digital `presentacion.html` de esta carpeta.
2. Entrega al alumnado `index.html`, `main.css` y `app.alumnos.js` (renombrado a `app.js`).
3. Guía la resolución de los Hitos 1 a 6 de forma incremental comprobando la consola del navegador.

[HITO 1] Banco de Palabras y Variables de Estado

> Definimos el vocabulario y las variables globales que retendrán en memoria el estado completo de la partida:

```
// Banco de palabras fijas en mayúsculas
const PALABRAS = ['JAVASCRIPT', 'VARIABLE', 'FUNCION', 'ARRAY', 'OBJETO', 'BUCLE'];
const MAX_FALLOS = 6;

let palabraSecreta = '';    // Palabra a adivinar en la partida (ej: "VARIABLE")
let letrasAdivinadas = [];  // Array con guiones y aciertos: ['_', 'A', '_']
let letrasUsadas = [];      // Array con letras ya probadas: ['E', 'A', 'Z']
let fallos = 0;             // Contador numérico de fallos cometidos (0 a 6)
let juegoTerminado = false; // Bandera booleana de control
```

[HITO 2] Selección de Elementos del DOM y Partes SVG

Guardamos en constantes las referencias a los elementos visuales una sola vez para no repetir consultas con `getElementById`:

```
const palabraDisplay = document.getElementById('palabra-display');
const inputLetra = document.getElementById('input-letra');
const btnProbar = document.getElementById('btn-probar');
const letrasUsadasDisplay = document.getElementById('letras-usadas-display');
const vidasRestantes = document.getElementById('vidas-restantes');
const mensajeEstado = document.getElementById('mensaje-estado');
const btnReiniciar = document.getElementById('btn-reiniciar');

// Array indexado con las 6 partes corporales del SVG
const partesAhorcado = [
  document.getElementById('parte-cabeza'),     // Índice 0 → Fallo 1
  document.getElementById('parte-cuerpo'),     // Índice 1 → Fallo 2
  document.getElementById('parte-brazo-izq'),  // Índice 2 → Fallo 3
  document.getElementById('parte-brazo-der'),  // Índice 3 → Fallo 4
  document.getElementById('parte-pierna-izq'), // Índice 4 → Fallo 5
  document.getElementById('parte-pierna-der')  // Índice 5 → Fallo 6
];
```

> **¿Por qué un array de elementos DOM?**
>
> Agrupar las partes del muñeco en un array permite acceder a la pieza exacta en tiempo constante simplemente con `partesAhorcado[fallos - 1]`, evitando una estructura `switch (fallos)` innecesariamente larga.

[HITO 3] La Función `iniciarJuego()`

> Esta función se encarga de resetear la memoria de la partida, seleccionar una nueva palabra aleatoria y restaurar el dibujo a su estado inicial:

```
function iniciarJuego() {
  // 1. Selección aleatoria de la palabra secreta
  const indiceAleatorio = Math.floor(Math.random() * PALABRAS.length);
  palabraSecreta = PALABRAS[indiceAleatorio];

  // 2. Creación del array de guiones con la longitud exacta
  letrasAdivinadas = [];
  for (let i = 0; i < palabraSecreta.length; i++) {
    letrasAdivinadas.push('_');
  }

  // 3. Reset de variables de control
  letrasUsadas = [];
  fallos = 0;
  juegoTerminado = false;

  // 4. Ocultar todas las partes del ahorcado quitando la clase 'visible'
  partesAhorcado.forEach(parte => {
    parte.classList.remove('visible');
  });
  caraDerrota.classList.remove('visible');

  // 5. Restaurar estado de controles y autoenfoque
  inputLetra.disabled = false;
  btnProbar.disabled = false;
  inputLetra.value = '';
  inputLetra.focus();

  // 6. Refrescar interfaz
  actualizarPantalla();
  mostrarMensaje('¡Nueva partida iniciada! Escribe una letra.', 'normal');
}
```

Puntos Clave Explicados para el Alumnado

El algoritmo Math.random()

`Math.random()` devuelve un número decimal entre 0 y 0.999... Al multiplicarlo por la longitud del array (ej. 6) y aplicar `Math.floor()`, obtenemos un entero exacto entre 0 y 5.

El bucle for de guiones

Si `palabraSecreta = "HTML"` (longitud 4), el bucle se ejecuta 4 veces insertando `'_'`, resultando en `['_', '_', '_', '_']`.

[HITO 4] La Función `procesarLetra()`

> Es el corazón algorítmico de la aplicación: valida la entrada del usuario, registra la jugada y bifurca el flujo según si la letra acierta o falla:

```
function procesarLetra() {
  if (juegoTerminado) return;

  // 1. Obtener y sanitizar el texto
  const letra = inputLetra.value.trim().toUpperCase();
  inputLetra.value = '';
  inputLetra.focus();

  // 2. Validaciones preventivas
  if (letra === '' || !/^[A-ZÑ]$/.test(letra)) {
    mostrarMensaje('Introduce una sola letra válida (A-Z).', 'aviso');
    return;
  }

  if (letrasUsadas.includes(letra)) {
    mostrarMensaje(`La letra "${letra}" ya fue jugada.`, 'aviso');
    return;
  }

  // 3. Añadir a historial
  letrasUsadas.push(letra);

  // 4. Comprobación en la palabra secreta
  if (palabraSecreta.includes(letra)) {
    // ACIERTO: Recorremos para destapar TODAS las apariciones
    for (let i = 0; i < palabraSecreta.length; i++) {
      if (palabraSecreta[i] === letra) {
        letrasAdivinadas[i] = letra;
      }
    }
    mostrarMensaje(`¡Acierto! La letra "${letra}" está en la palabra.`, 'exito');
  } else {
    // FALLO: Incrementamos fallos y dibujamos la pieza SVG
    fallos++;
    dibujarSiguienteParte(fallos);
    mostrarMensaje(`¡Fallo! La letra "${letra}" no existe.`, 'error');
  }

  // 5. Actualizar display y verificar fin
  actualizarPantalla();
  verificarFinJuego();
}
```

Pregunta frecuente de alumnos: ¿Por qué no usar indexOf?

Porque `indexOf()` devuelve únicamente el índice del *primer* encuentro. Si la palabra es "PROGRAMACION", la letra 'A' aparece en las posiciones 5 y 8; el bucle `for` asegura descubrir ambas sin dejar letras ocultas.

[HITO 5] Actualización y Dibujo del Ahorcado

> Mantenemos la interfaz sincronizada mediante funciones con propósitos específicos y reutilizables:

```
function actualizarPantalla() {
  // Unimos los elementos con espacios para que se vea: "_ _ J _ S"
  palabraDisplay.textContent = letrasAdivinadas.join(' ');

  const vidas = MAX_FALLOS - fallos;
  vidasRestantes.textContent = vidas;
  vidasRestantes.classList.toggle('peligro', vidas ⩽ 2);

  letrasUsadasDisplay.textContent = letrasUsadas.length ? letrasUsadas.join(', ') : 'Ninguna aún';
}

function dibujarSiguienteParte(numeroFallo) {
  const indice = numeroFallo - 1;
  if (indice ⩾ 0 && indice < partesAhorcado.length) {
    partesAhorcado[indice].classList.add('visible');
  }
}
```

[HITO 6] Verificación de Fin de Partida

```
function verificarFinJuego() {
  // Condición 1: VICTORIA (¡ya no quedan guiones bajos!)
  if (!letrasAdivinadas.includes('_')) {
    juegoTerminado = true;
    mostrarMensaje(`🎉 ¡VICTORIA! Has descubierto "${palabraSecreta}".`, 'exito');
    finalizarPartida();
    return;
  }

  // Condición 2: DERROTA (se han cometido 6 fallos)
  if (fallos ⩾ MAX_FALLOS) {
    juegoTerminado = true;
    caraDerrota.classList.add('visible');
    palabraDisplay.textContent = palabraSecreta.split('').join(' ');
    mostrarMensaje(`💀 ¡GAME OVER! La palabra era "${palabraSecreta}".`, 'error');
    finalizarPartida();
  }
}

function finalizarPartida() {
  inputLetra.disabled = true;
  btnProbar.disabled = true;
  btnReiniciar.focus();
}
```

Solución Canónica Completa (app.solucion.js)

```
const PALABRAS = ['JAVASCRIPT', 'PROGRAMACION', 'VARIABLE', 'FUNCION', 'ARRAY', 'OBJETO'];
const MAX_FALLOS = 6;
let palabraSecreta = '', letrasAdivinadas = [], letrasUsadas = [], fallos = 0, juegoTerminado = false;

const palabraDisplay = document.getElementById('palabra-display');
const inputLetra = document.getElementById('input-letra');
const btnProbar = document.getElementById('btn-probar');
const letrasUsadasDisplay = document.getElementById('letras-usadas-display');
const vidasRestantes = document.getElementById('vidas-restantes');
const mensajeEstado = document.getElementById('mensaje-estado');
const btnReiniciar = document.getElementById('btn-reiniciar');
const partesAhorcado = ['cabeza', 'cuerpo', 'brazo-izq', 'brazo-der', 'pierna-izq', 'pierna-der']
  .map(id => document.getElementById(`parte-${id}`));
const caraDerrota = document.getElementById('parte-cara-derrota');

function iniciarJuego() {
  palabraSecreta = PALABRAS[Math.floor(Math.random() * PALABRAS.length)];
  letrasAdivinadas = Array(palabraSecreta.length).fill('_');
  letrasUsadas = []; fallos = 0; juegoTerminado = false;
  partesAhorcado.forEach(p => p.classList.remove('visible'));
  caraDerrota.classList.remove('visible');
  inputLetra.disabled = false; btnProbar.disabled = false; inputLetra.value = ''; inputLetra.focus();
  actualizarPantalla();
}

function procesarLetra() {
  if (juegoTerminado) return;
  const l = inputLetra.value.trim().toUpperCase();
  inputLetra.value = ''; inputLetra.focus();
  if (!l || !/^[A-ZÑ]$/.test(l) || letrasUsadas.includes(l)) return;
  letrasUsadas.push(l);
  if (palabraSecreta.includes(l)) {
    for (let i = 0; i < palabraSecreta.length; i++) if (palabraSecreta[i] === l) letrasAdivinadas[i] = l;
  } else {
    fallos++;
    if (partesAhorcado[fallos - 1]) partesAhorcado[fallos - 1].classList.add('visible');
  }
  actualizarPantalla();
  verificarFinJuego();
}

btnProbar.addEventListener('click', procesarLetra);
inputLetra.addEventListener('keydown', e => { if (e.key === 'Enter') procesarLetra(); });
btnReiniciar.addEventListener('click', iniciarJuego);
iniciarJuego();
```

¡Enhorabuena! Has completado el análisis de la Versión 1 Básica Lineal. 🎮 🎓